

Prompt M4.3Live

Moteur M4.3 pilotable en direct, à la recherche d'un effet rebond

Lignée M4.3Live — succède à M4.3 (`m4_3_credit_soc`, lu comme référence structurelle, pas une dépendance destinée à une instance Claude Opus 5)

17 août 2026 — révision 2

Destinataire : une instance Claude Opus 5 (Claude Code), à qui ce document est confié comme spécification de tâche complète et autonome. Ce n'est pas un brouillon à discuter avant de commencer : les décisions de fond sont prises ci-dessous ; les points explicitement laissés ouverts (§11) sont à trancher et documenter toi-même, pas à retourner à l'utilisateur avant d'avoir essayé.

*Version : 17 août 2026, révision 2. Lignée : succède à M4.3 (`m4_3_credit_soc/`, lu comme référence structurelle — **pas une dépendance**, §3.5) mais n'est plus une extension au sens strict : l'institution de principal elle-même change de nature (§3). Convention de dossier `m4_3live_credit_soc/` inchangée.*

Provenance de cette révision : ce document remplace la révision 1 du 17 août (matin). Il intègre (a) les annotations manuscrites de l'utilisateur portées sur le PDF de la révision 1 (15 annotations, dont une vide, entre 12h44 et 16h21) et (b) le rapport `rapport_architecture_offline_online.pdf` (17 août, après-midi, copié dans ce dossier pour pérennité) que ces annotations ont motivé. **Une annotation est explicitement dépassée par deux autres, postérieures de 25 à 70 minutes** : à 12h57 l'utilisateur note « on reste en arithmétique tout du long » à propos de `target_rule` figé au lancement — lecture encore valide sur ce point précis (§2) — mais à 13h33 et 14h09 il redéfinit complètement ce que « arithmétique » signifie (§3). Ne pas lire la première annotation comme figeant l'ancienne formule littérale $(K_\ell - K_b)/2$ en toute circonstance.

Table des matières

0. Lectures préalables obligatoires, et pièges à ne pas suivre	1
1. Objectif scientifique : l'effet rebond, à observer pour l'instant, pas à expliquer	2
2. Portées d'intervention : sémantique exacte, trois portées, pas deux	3
3. L'institution de principal : maximiser la puissance d'extraction jointe	4
4. Architecture « en direct » : session, boucle de pas, journal d'interventions	7
5. IHM HTML — un outil de recherche interne, pas un produit	8
6. Intégration avec <code>simulation_lab</code> : isolation, non-régression	9
7. Protocole expérimental pour la question du rebond	9
8. Tests exigés	9

9. Livrables	10
10. Arborescence attendue	10
11. Décisions laissées ouvertes, à trancher et documenter toi-même	11
12. Consignes de méthode de travail, pour toi (Opus 5)	11
Sous-agents : subdiviser intelligemment, pas systématiquement	12

0. Lectures préalables obligatoires, et pièges à ne pas suivre

Avant d'écrire une ligne de code, lire dans cet ordre :

- `m4_3_credit_soc/m4_3/model.py` (référence structurelle, ~800 lignes, **pas une dépendance à importer**, §3.5) — c'est le fichier dont ce prompt cite les numéros de ligne ci-dessous ; si le fichier a changé depuis, revérifier les citations, ne pas les croire sur parole.
- `m4_3_credit_soc/m4_3/io.py` et `m4_3_credit_soc/report/rapport_final.md` (contexte scientifique de M4.3 : pourquoi `loan_events`, pourquoi la règle de principal historique était arithmétique).
- `simulation_lab/contracts.py`, `simulation_lab/jobs.py`, `simulation_lab/web/app.py`, `docs/README_simulation_lab.md` — l'outil d'orchestration existant, dont ce prompt étend l'usage sans le casser.
- `CLAUDE.md` (racine du dépôt) — règles de travail du dépôt.
- `m4_3live_credit_soc/prompts/ rapport_architecture_offline_online.pdf` — étude commandée par l'utilisateur sur le calcul efficace, à l'échelle de millions de résolutions, du transfert optimal introduit en §3. Lecture obligatoire avant d'écrire une ligne du §3 : ce prompt en reprend les équations et l'architecture recommandée sans les redémontrer.
- Optionnel mais utile, pas bloquant : les rapports des autres lignées du dépôt (`m4_credit_soc/`, `m4b_credit_soc_mini/`, `m4_2_credit_soc/`, `m4_2b_credit_soc/`, chacun avec son `report/`) donnent une vision du fonctionnement intrinsèque de cette classe de modèle au-delà de M4.3 (ex. tension entre taille moyenne et taille autarcique du système, effet d'une multiplication scalaire des paramètres) — ne pas redécouvrir ce qui y est déjà documenté.

Deux passages de `CLAUDE.md` et de `docs/README_simulation_lab.md` sont obsolètes et vont t'induire en erreur si tu les suis sans vérifier :

1. `CLAUDE.md` désigne `m4b_credit_soc_mini/` comme « moteur actif » et donne `random.Random(seed)` comme convention RNG. **Faux pour ce travail** : le moteur de référence structurelle est `m4_3_credit_soc/m4_3/model.py`, et son RNG est `numpy.random.default_rng(seed)` (`Simulation.__init__`, `m4_3/model.py:525`) — convention à reproduire dans le fork indépendant de M4.3Live (§3.5).
2. `docs/README_simulation_lab.md` liste `web/templates/index.html` et donne « `stdlib + matplotlib` » comme dépendances. **Faux** : les templates actuels sont `launch.html/results.html` (voir `simulation_lab/web/app.py:50-53`), et `numpy/scipy` sont explicitement autorisés en plus de la `stdlib`. Pas de nouvelle dépendance externe (pas de `Flask/FastAPI/websockets` tiers) sans accord explicite de l'utilisateur — rester sur `http.server/ThreadingHTTPServer`.

Règle de confiance à 95 % du dépôt (`CLAUDE.md`) : si tu n'es pas certain à 95 % d'un fait sur ce projet, le signaler explicitement (fait observé / inférence / hypothèse / incertitude). Citer systématiquement `fichier.py:ligne`. Ne jamais inventer un comportement du moteur non vérifié dans le code.

1. Objectif scientifique : l'effet rebond, à observer pour l'instant, pas à expliquer

Le moteur M4.3 produit, par pas de temps, une production agrégée `prod_tot` (`Simulation.series`, calculée `model.py:563-568` : $A \cdot K^\gamma$ par entité, sommée — inchangé quelle que soit l'institution de principal, §3). La « puissance d'extraction » d'une entité est portée par deux paramètres de sa fonction de production/rendement, A et γ — ce ne sont **pas** des leviers interchangeable :

- **A est le levier primaire.** Il multiplie linéairement la production et le rendement marginal — une hausse de A augmente la production pour toute capitalisation, sans ambiguïté de sens.
- **γ est un levier secondaire, à interprétation non monotone.** `Config.__post_init__` (`model.py:86-87`) contraint $\gamma \in]0, 1[$. Pour $K < 1$, augmenter γ *diminue* K^γ . Toute conclusion sur un effet rebond via γ doit signaler cette non-monotonie plutôt que traiter γ comme un simple cadran de puissance.

Définition de travail de l'effet rebond pour ce programme : une augmentation de A (et/ou γ , avec la réserve ci-dessus) appliquée à une partie de la population — via la portée `fraction` ou la portée `new` du §2, pas nécessairement un sous-ensemble figé une fois pour toutes — se traduit, sur `prod_tot` agrégé du système entier, par une réponse non proportionnelle à l'augmentation locale — voire par une baisse — par comparaison à l'état qu'aurait eu le système en poursuivant **sans** ce changement dynamique de paramètres (une simulation de contrôle de même graine, §7), et par comparaison à une intervention de même amplitude appliquée à la population entière (portée `all`). Ce n'est pas un critère binaire prêt à l'emploi : à toi de figer une statistique précise (élasticité de `prod_tot` à l'intervention, fenêtre d'observation post-intervention, agrégation multi-graines) avant de lancer la campagne de mesure, et de la documenter.

L'objectif de cette itération du programme est d'OBSERVER l'effet rebond, pas de l'expliquer ni de le justifier (décision explicite de l'utilisateur). Trois canaux plausibles de propagation d'une hausse locale — pour information, comme grille de lecture, **pas comme livrable obligatoire** :

1. **Production directe** : les entités boostées produisent plus, mécaniquement.
2. **Accumulation de capital** : la production s'ajoute au capital de l'entité (`model.py:567`, `population.K[entity] += produced`) avant la phase de marché du même pas — une entité boostée entre dans le marché du crédit avec un K plus élevé, ce qui change sa position de prêteuse ou d'emprunteuse.
3. **Effet sur les échanges** : une entité boostée change le calcul du transfert (§3) et, si la piste du §3.4 aboutit, du taux, dans chaque paire où elle apparaît — donc indirectement le risque de faillite des autres entités.

Si une décomposition sur ces trois canaux ressort naturellement de l'analyse, la documenter dans le rapport de résultats (§9) comme analyse complémentaire est bienvenu — mais ne pas la traiter comme un critère de succès, et ne pas retarder la campagne de mesure du §7 pour l'obtenir. Le résultat attendu est un verdict d'observation (§7) : effet rebond présent / absent / non tranchable avec le budget disponible.

2. Portées d'intervention : sémantique exacte, trois portées, pas deux

La demande de l'utilisateur (« changer les variables pour toutes les entités ou seulement pour les nouvelles ») définit deux portées ; ce programme en exige une troisième pour que l'expérience du §7 soit interprétable. Les trois portées, comme fonctionnalité générale du système, applicables (en principe) à n'importe quel paramètre numérique de `Config` :

- **`all` (toutes les entités)** : la nouvelle valeur s'applique immédiatement à toutes les entités actuellement vivantes *et* à toutes les entités futures — rétroactif et prospectif.
- **`new` (nouvelles entités seulement)** : la nouvelle valeur ne s'applique qu'aux entités nées

après l'intervention ; les entités déjà vivantes gardent leur valeur d'origine **pour toujours** — sémantique de vintage/cohorte technologique (le capital « embarqué » la technologie de sa date de naissance, terminologie standard en économie de la croissance ; le mentionner dans le rapport de conception, §9).

- **fraction (fraction φ de la population vivante, tirage aléatoire)** : au moment t_0 de l'intervention, tirer une fraction $\varphi \in]0, 1]$ (ou une liste explicite d'identifiants, pour le débogage) des entités **actuellement vivantes** et leur assigner la nouvelle valeur, une seule fois. **Ce que deviennent les naissances postérieures à t_0 (question explicitement posée par l'utilisateur en annotation) : une intervention fraction ne modifie PAS la valeur par défaut utilisée aux naissances futures.** Seules les entités tirées à t_0 sont affectées ; les entités nées après t_0 reçoivent le défaut alors en vigueur (celui d'avant l'intervention, sauf si une intervention **all/new** distincte est émise séparément sur le même paramètre). Raison de ce choix, à ne pas perdre en implémentant : c'est ce qui garde l'intensité de traitement **fixe** au moment de la mesure — la raison d'être même de la portée **fraction**. Si **fraction** modifiait aussi le défaut, elle se comporterait comme **new** plus un tirage ponctuel, et l'intensité dériverait à nouveau dans le temps.

Portée requise pour l'expérience du §7 : **new** a une intensité de traitement qui dérive dans le temps (avec λ constant, la part boostée de la population croît de 0 vers un régime permanent au fil des naissances/morts, donc mesurer une élasticité contre une intensité de traitement mouvante n'a pas de sens) ; **fraction** fixe l'intensité au moment de la mesure — c'est la portée utilisée pour le protocole principal, **all** et **new** restent des modes d'exploration secondaires (§7).

Où la portée a un sens, et où elle n'en a pas — à ne pas deviner en silence :

- A et γ (par entité, §3) : les trois portées ont un sens et doivent toutes être implémentées. **Attention** (mise en garde explicite de l'utilisateur) : modifier A ou γ pour une cohorte change son rendement marginal, donc son échelle de capital naturelle (l'échelle autarcique déjà documentée dans la lignée M4.2/M4.2B — voir `m4_2_credit_soc/` ou `m4_2b_credit_soc/` si la formule doit être retrouvée). Faire varier A/γ pour une cohorte **sans** ajuster K_0 en proportion peut changer la tension du système (taille relative à l'échelle autarcique) en même temps que l'extraction, ce qui confondrait l'expérience du §7. Ce prompt ne tranche pas si K_0 doit être co-varié automatiquement avec une intervention A/γ — c'est une décision de protocole à prendre et à documenter explicitement avant la campagne du §7 (liste des décisions ouvertes, §11), pas un détail à laisser implicite.
- K_0 (capital de naissance) : n'a de sens qu'au moment de la naissance — il n'existe rien à modifier « rétroactivement » sur une entité déjà née. **all** et **new** sont donc **identiques par construction** pour K_0 ; **fraction** n'a pas de sens pour K_0 seul (pas d'implémentation requise, sauf si utilisé conjointement avec A/γ comme au paragraphe précédent). Documenter cette dégénérescence dans l'IHM (ne pas la cacher en désactivant silencieusement un sélecteur sans explication) et dans le rapport de conception.
- **lam, delta, sigma, rho, eta_beta, eta_n_ref** : ce sont des paramètres de population/marché, pas des attributs d'entité — **new** doit être un alias explicite de **all** (documenté comme tel dans l'IHM, pas masqué), **fraction** n'a pas de sens (pas d'implémentation requise). Ceci ne les exclut pas comme axes secondaires de l'**expérience** du §7 elle-même — des jeux différents de ces paramètres, donc des régimes de population/marché différents, peuvent être intéressants à observer pour la question du rebond (remarque de l'utilisateur) ; seule l'extension de la sémantique de portée **fraction/new** à ces paramètres est hors mandat.
- **target_rule** : dans M4.3Live, il n'existe plus qu'une seule institution de principal (§3) — la bascule arithmétique/géométrique de M4.2B/M4.3 n'est pas reprise (M4.3Live est indépendant de ce moteur, §3.5). Le champ peut être conservé dans **Config** pour usages futurs mais n'a, pour ce prompt, aucune valeur alternative à exposer ; **exclu de l'intervention en direct** pour la même raison que pour les autres choix institutionnels figés : rien dans la question du rebond (§1, §7) n'exige de changer d'institution en cours de run.

Minage/synchronisation avec la boucle de pas : une intervention arrive de manière asynchrone (requête HTTP) pendant qu'un thread exécute `Simulation.step()` en boucle. Elle doit être **mise en file d'attente** et appliquée en **tout début du prochain appel à `step()`**, **avant les naissances** — jamais pendant un `step()` en cours. L'intervention effectivement appliquée doit être journalisée avec le t réel auquel elle a pris effet. Le tirage `fraction` doit utiliser le générateur RNG de la simulation (`self.rng`) pour rester déterministe et rejouable — pas un générateur séparé. **La mise en pause du direct ne doit consommer aucun tirage RNG** : mettre en pause puis reprendre et exécuter N pas doit produire une trajectoire strictement identique à N pas exécutés sans pause — invariant à tester (§8).

3. L'institution de principal : maximiser la puissance d'extraction jointe

Ce point remplace intégralement le §3 de la révision précédente de ce prompt. L'utilisateur abandonne la règle arithmétique littérale de M4.3 ($q_A = (K_\ell - K_b)/2$, `m4_3_credit_soc/m4_3/model.py`: 269–291) comme institution première de M4.3Live : cette formule n'était qu'un cas particulier d'un problème plus général, celui du montant à échanger entre deux entités pour **maximiser leur puissance de production jointe immédiatement après l'échange**. Le rapport `rapport_architecture_offline_online.pdf` (§0, lecture obligatoire) pose et résout ce problème :

$$\max_{-x \leq \delta \leq y} a(x + \delta)^\alpha + b(y - \delta)^\beta, \quad 0 < \alpha, \beta < 1, \quad a, b > 0 \quad (\text{rapport §1, éq.1})$$

avec $x = K_b$ (capital de l'emprunteuse), $y = K_\ell$ (capital de la prêteuse), $(a, \alpha) = (A_b, \gamma_b)$, $(b, \beta) = (A_\ell, \gamma_\ell)$ (rapport §2, éq.8). En posant $C = x + y$ (la somme conservée par l'échange), la solution s'écrit $\delta^* = C\lambda^* - x$ pour un certain $\lambda^* \in [0, 1]$ (rapport §1, éq.2). C'est le calcul de λ^* (ou du capital optimal $h(C) = C\lambda^*$ attribué à l'emprunteuse) qui pose un problème de coût selon le régime.

3.1 Trois régimes, un seul principe institutionnel

Régime	Condition	Calcul	Coût
(a) homo-gène	$A_\ell = A_b$ et $\gamma_\ell = \gamma_b$	$\delta^* = (K_\ell - K_b)/2$, la formule historique de M4.3	trivial
(b) exposants égaux	$\gamma_\ell = \gamma_b$, $A_\ell \neq A_b$ possible	forme fermée exacte, éq. (1) ci-dessous	trivial (un log/exp par couple de technologies, jamais par transaction)
(c) exposants différents	$\gamma_\ell \neq \gamma_b$	pas de forme fermée — machinerie du rapport, §3.2	dépend du cache

Formule fermée du régime (b) (rapport §3, éq.14) :

$$\lambda^* = \frac{A_b^{1/(1-\gamma)}}{A_b^{1/(1-\gamma)} + A_\ell^{1/(1-\gamma)}} \quad (1)$$

Point à ne pas manquer, cause probable de la confusion la plus fréquente sur ce point : le régime (b) n'est **pas** la formule historique. λ^* n'est égal à $1/2$ que si $A_\ell = A_b$ en plus de $\gamma_\ell = \gamma_b$ (régime a) — vérifier : à $A_b = A_\ell = A$, $A_b^{1/(1-\gamma)} = A_\ell^{1/(1-\gamma)}$ et $\lambda^* = 1/2$ exactement, ce qui redonne $\delta^* = (K_\ell - K_b)/2$; dès que $A_b \neq A_\ell$, $\lambda^* \neq 1/2$. Une intervention qui ne modifie que

A (§1, levier primaire) fait donc déjà sortir toute paire mixte du régime (a) vers le régime (b), avec un partage **pondéré** du capital — « si les deux gammas sont égaux, le montant est aisément calculable » (annotation de l'utilisateur) signifie « calculable en forme fermée », pas « égal à la moitié de l'écart ».

3.2 Régime (c) : architecture hot/warm/cold du rapport

Le rapport résout le régime (c) en exploitant une propriété structurelle de M4.3Live, pas générique : (A_i, γ_i) prend un petit nombre de valeurs discrètes **exactes** (des « technologies »), pas des perturbations continues (rapport §6.1) — la bonne opération est de conditionner exactement sur le couple **ordonné** de technologies (émettrice, receveuse), pas d'approximer dans un espace à 6 paramètres continus (rapport §6.3 : un surrogate générique à 6D construit avant l'inspection du cas réel s'est avéré inutile une fois cette structure exploitée — ne pas répéter cette impasse). À couple de technologies fixé, toute la non-linéarité tient dans une seule dimension $C = x + y$ — le Hessien de δ^* est exactement de rang un (rapport §6, éq.29-32), indépendamment des corrélations entre x et y .

Architecture exigée (rapport §11, « architecture recommandée » — suivre sauf justification écrite d'un meilleur choix dans le rapport de conception, §9) :

- **Indexation par technologie, pas par flottant.** Chaque (A_i, γ_i) distinct reçoit un identifiant entier à sa première apparition ; une matrice dense `kernel[s_émettrice][s_receveuse]` route un couple en $O(1)$ (rapport §7.1).
- **Chemin chaud** : régime (b) dès que $\gamma_{\text{émettrice}} = \gamma_{\text{receveuse}}$ — aucune table, formule fermée (§3.1).
- **Chemin tiède** : couple de technologies nouveau, ou table en cours de construction — noyau sans table (une étape de Newton depuis $z_0 = -2t$, rapport §4.2 éq.21, ou deux tours de point fixe) le temps d'observer assez d'occurrences.
- **Chemin froid** : Newton robuste (rapport §4, formulation exacte en (t, u, z) , éq.17) pour la première occurrence d'un couple, un C hors du domaine couvert par la table, ou un audit aléatoire.
- **Construction paresseuse d'une LUT 1D $h(C)$** par couple de technologies non trivial, après un seuil d'amortissement **mesuré sur la machine cible** (indicatif du rapport : ~ 1800 usages face à Newton exact sur son banc scalaire, §8.1 — **à recalibrer**, ce chiffre vient d'un benchmark synthétique, pas de ce moteur) ; interpolation linéaire ou cubique de Hermite selon la tolérance requise (rapport §7.2 pour le pseudo-algorithme complet, à reprendre).
- **Aucun tirage aléatoire pendant la construction/extension d'une table** — la compilation peut s'exécuter en tout début du pas où une intervention est appliquée (même point de synchronisation que §2) ; elle change la durée murale du pas, jamais la trajectoire (rapport §7.2, dernier paragraphe).
- **La LUT 2D globale n'est pas nécessaire** dans M4.3Live : elle resterait le baseline si les exposants redevenaient continus, mais le support discret transforme chaque cellule en problème 1D moins coûteux (rapport §11, conclusion).

Contrainte institutionnelle à fixer explicitement, pas seulement numérique (rapport §2, éq.9) : le marché ne prête aujourd'hui que de la plus riche vers la plus pauvre (`K[lender] >= K[borrower]`, `m4_3_credit_soc/m4_3/model.py`: 357-361, comportement structurel hérité sans changement, §3.3). Deux choix possibles pour la borne supérieure de δ^* : (i) autoriser le transfert jusqu'à l'optimum général $h(C)$, qui peut dépasser l'égalisation des capitaux ; (ii) le plafonner à l'égalisation $((K_\ell - K_b)/2)$ dans un cas homogène, une borne équivalente en général). Ce prompt ne tranche pas — décision à documenter dans le rapport de conception, et listée en §11.

3.3 Hérité sans changement

- Le sens du prêt (plus riche vers plus pauvre), le pool $k \equiv 2$, les faillites cancel+destroy, l'ordre des phases — tout hérité de M4.3 comme référence structurelle.

- La formule de production $A \cdot K^\gamma$ par entité — §1.
- A_i et γ_i par entité sont deux listes parallèles sur `Population`, fixées à la naissance, jamais retirées ni réutilisées après une mort — même convention que les listes existantes de `Population` (`K`, `alive`, `birth`, ...) dans le moteur de référence.
- `_pair_rate` (`m4_3_credit_soc/m4_3/model.py`: 258-266, $m = A\gamma K^{\gamma-1}$, $rate = \sqrt{m_\ell m_b}$) reste le mécanisme de taux **par défaut** de M4.3Live — voir §3.4, ce n'est pas remplacé par défaut, et sa généralisation par entité (chaque côté utilisant son propre A_i, γ_i) est directe et sans ambiguïté, contrairement au problème du principal.

3.4 Le taux d'intérêt : piste ouverte, pas un prérequis bloquant

Une annotation de l'utilisateur propose d'aller plus loin que le rapport : au lieu de garder `_pair_rate` inchangé, faire en sorte que « les entités se partagent le surplus dû à leur coopération juste après leur contrat », via un paramètre de partage p qui « remplace m ». Le rapport, lui, ne couvre **que** le problème du montant (son §2 dit explicitement que remplacer le principal par ce transfert optimal serait « une nouvelle règle de principal », distincte du calcul du taux) — cette extension au taux est une piste de l'utilisateur à explorer, **pas un résultat déjà établi**, et ce prompt ne la considère pas comme résolue.

Ce qu'un candidat p devrait produire, précisément. Le carnet de prêts a besoin d'un taux scalaire `rate` tel que $due = principal \times rate$ à chaque pas, perpétuellement (`m4_3_credit_soc/m4_3/model.py`: 160 pour l'agrégat `due`, `model.py`: 572-594 pour le service payé chaque pas — mécanique à reproduire à l'identique dans le fork, §3.5). Le surplus coopératif, lui, est une quantité **ponctuelle**, calculée une seule fois au moment du contrat :

$$\Delta = [A_b(x + \delta^*)^{\gamma_b} + A_\ell(y - \delta^*)^{\gamma_\ell}] - [A_b x^{\gamma_b} + A_\ell y^{\gamma_\ell}].$$

Le problème réel n'est pas « comment partager Δ » (un $p \in [0, 1]$ suffit à répondre à cette partie) — **c'est que transformer une quantité ponctuelle Δ en un taux perpétuel r n'est pas un problème bien posé sans une hypothèse supplémentaire** (un horizon implicite, un taux d'actualisation, une convention de rente perpétuelle...). C'est cette hypothèse manquante, pas le partage lui-même, qui est la vraie question ouverte à trancher.

Ce que ce prompt exige si cette piste est explorée :

- Documenter explicitement l'hypothèse choisie pour transformer $(\Delta, p, principal)$ en un taux r — pas un choix implicite enterré dans le code.
- Vérifier que $r > 0$ et que ce taux ne casse pas mécaniquement la solvabilité au pas suivant : `due` entre dans le calcul **avant** la dépréciation (`model.py`: 572-594) — un taux dérivé du surplus peut être très supérieur à l'ancien taux de rendement marginal, et un pic de faillites causé par cet artefact numérique n'aurait rien à voir avec la question du rebond (§1). Documenter cette vérification, pas seulement l'effectuer.
- `_pair_rate` (§3.3) **reste le mécanisme opérationnel par défaut** tant que cette piste n'est pas terminée et validée par les deux points ci-dessus. La question du rebond (§7) est mesurable avec le mécanisme de taux existant — ne pas bloquer tout le programme sur p . Traiter ce point comme un chantier de conception **parallèle**, à documenter dans le rapport de conception (§9) qu'il ait abouti ou non, pas comme un préalable au reste du prompt.

3.5 Fork indépendant, pas un import

M4.3Live doit être indépendant des autres modèles du dépôt (décision explicite de l'utilisateur) — ce qui referme la question laissée ouverte dans la révision précédente de ce prompt (« import direct vs fork »). Ne **pas** importer `m4_3_credit_soc.m4_3` comme dépendance : écrire le moteur de M4.3Live comme un paquet autonome dans le nouveau dossier (§10), qui lit `m4_3_credit_soc/m4_3/model.py` comme référence structurelle (mêmes conventions : `Population/LoanBook` à listes parallèles jamais réindexées, `RNG numpy.random.default_rng(seed)`, ordre des phases) sans en dépendre au sens Python du terme.

3.6 Parité avec M4.3 : souhaitable si simple, plus une exigence

Ce point change par rapport à la révision précédente de ce prompt (annotation explicite de l'utilisateur) : une correspondance bit-à-bit avec M4.3 n'est **plus requise**. Si elle s'obtient simplement dans le régime (a) (§3.1) — c'est-à-dire si le chemin homogène du fork produit exactement $(K_\ell - K_b)/2$ par construction plutôt que par coïncidence numérique — c'est un plus à documenter, pas un objectif à poursuivre activement. Les régimes (b) et (c) n'ont de toute façon aucun analogue dans M4.3 littéral (qui n'implémentait que le cas homogène) : aucune parité n'est même définie pour eux.

4. Architecture « en direct » : session, boucle de pas, journal d'interventions

- **Boucle de simulation** : un thread dédié appelle `Simulation.step()` en boucle à une cadence pilotable (lecture/pause, pas-à-pas, vitesse), avec une porte (`threading.Event` ou équivalent) pour la pause — voir l'invariant RNG du §2. La file d'attente d'interventions est un objet partagé protégé par verrou, purement stdlib (`threading.Lock/queue.Queue`), dans l'esprit de `simulation_lab/jobs.py` (déjà `threading`-based) plutôt que le modèle `multiprocessing.Process` de `_run_single_model_worker` (`simulation_lab/jobs.py:24-51`) — un sous-processus séparé rendrait le partage d'état (population potentiellement jusqu'à `pop_max=30000` entités) coûteux à faire transiter par `Queue` à chaque pas ; documenter ce choix dans le rapport de conception.
- **API de pilotage unique, partagée entre l'IHM HTML et un pilote sans tête** : la fonction qui applique une intervention (portée, paramètre, valeur, éventuel t_0/φ) doit être un point d'entrée Python callable aussi bien depuis les gestionnaires HTTP `do_POST` de l'IHM que depuis un script headless — la campagne de mesure du §7 nécessite plusieurs graines et un plan d'intervention identique, rejouable par script.
- **Flux d'état vers le navigateur** : suivre la convention de sondage déjà en place dans `simulation_lab` (`GET /api/jobs/<id>` interrogé périodiquement, cf. `simulation_lab/web/static/app.js`) — endpoint JSON renvoyant l'état courant, sondé à intervalle fixe côté client. Pas de nouvelle dépendance (pas de websocket tiers).
- **Journal d'interventions, obligatoire, artefact du run** : chaque intervention effectivement appliquée (t effectif, paramètre, portée, ancienne valeur, nouvelle valeur, et pour **fraction** les identifiants effectivement tirés) est écrite dans un fichier à côté des autres artefacts du run. **Propriété exigée et testée (§8)** : rejouer un run en mode headless à partir de (seed, paramètres initiaux, journal d'interventions) doit reproduire une trajectoire strictement identique à l'originale.
- **Statuts de fin de simulation** ("extinction"/"explosion", `model.py:678-681`) : la boucle en direct doit s'arrêter proprement sur ces statuts et l'afficher clairement côté IHM.
- **Reprise depuis un run M4.3 existant** (demande explicite de l'utilisateur) : pouvoir, à partir d'un run M4.3 déjà stocké dans `simulation_lab_data/runs/<run_id>/` (paramètres et graine dans ses métadonnées, §6), lancer une session M4.3Live à partir d'un pas t_0 arbitraire (1000, 2000, ...) de ce run. **Impossibilité à énoncer clairement, pas à contourner en silence** : M4.3Live étant un fork avec une institution différente (§3), rejouer le run stocké avec le moteur M4.3Live ne reproduit sa trajectoire **que** dans le régime homogène (§3.1a), et seulement si les chemins flottants concordent — ce n'est pas garanti par construction (§3.6). Exiger donc une **vérification de divergence obligatoire à chaque reprise** : rejouer depuis (seed, paramètres) jusqu'à t_0 avec le moteur M4.3Live, comparer la série obtenue à la série stockée du run original, rapporter l'écart maximal observé — ne jamais présenter une reprise comme identique au run d'origine sans avoir mesuré cet écart.
- **Snapshot complet à t_0 , pour brancher plusieurs sessions** (demande explicite de l'utilisateur) : une fois un run amené à t_0 (reprise depuis M4.3 ci-dessus, ou session M4.3Live

fraîche), sauvegarder un snapshot complet et picklable de l'état (population, carnet de prêts, RNG, caches du §3.2) pour pouvoir relancer plusieurs sessions différentes (contrôle, un ou plusieurs traitements) depuis ce même t_0 sans re-simuler depuis $t = 0$ à chaque fois — sert directement le protocole apparié du §7.

5. IHM HTML — un outil de recherche interne, pas un produit

Nouvelle page (ex. `/live`), servie par extension de `simulation_lab/web/app.py` ou par un serveur dédié isolé (à trancher, §11) mais dans les deux cas construite pour ne pas régresser l'existant (§6). Contenu minimal exigé :

- graphiques en direct des séries clés (`prod_tot`, `K_tot`, `pop`, `defaults`, et une vue de la dispersion de A/γ dans la population — au minimum moyenne par cohorte née avant/après chaque intervention) ;
- panneau de contrôle : paramètre, nouvelle valeur, portée (`all/new/fraction` avec champ φ quand pertinent, §2), avec les dégénérescences du §2 explicitement affichées (pas de sélecteur qui ment sur ce qu'il fait) ;
- contrôles de lecture : `play/pause/pas-à-pas/vitesse` ;
- journal visible des interventions appliquées (t , paramètre, portée, valeurs).

C'est un outil de recherche interne, que l'utilisateur veut pouvoir manipuler lui-même en complément d'un travail autonome de l'agent (remarque explicite de l'utilisateur) : priorité à la clarté et à l'exactitude des chiffres affichés sur le raffinement visuel. Réutiliser le style existant (`simulation_lab/web/static/app.css`) plutôt qu'en construire un nouveau. Ne pas sur-investir en design.

6. Intégration avec `simulation_lab` : isolation, non-régression

`simulation_lab/` reste fonctionnel pour ses usages actuels (lancement simple/`batch`, `/launch`, `/results`, CLI `list-models/run/batch/list-runs`) après ce travail — vérifier au sens propre (relancer ces commandes, pas seulement lire le code) avant de considérer la tâche terminée. Isoler le nouveau code plutôt que modifier en place les chemins existants de `contracts.py/jobs.py/runs/` — par exemple un module `simulation_lab/live/` séparé, avec ses propres routes (`/live`, `/api/live/*`) ajoutées à `SimulationLabHandler` (`simulation_lab/web/app.py:45-193`) sans toucher aux branches existantes de `do_GET/do_POST`. La fonctionnalité de reprise (§4) lit les runs déjà stockés via `simulation_lab.runs.storage.RunStorage` (métadonnées, seed, paramètres) — en lecture seule, sans modifier le format de stockage existant. Si un adaptateur `BaseSimulationModel` (`contracts.py`) a un sens pour exposer une version *batch* (sans direct) du moteur étendu via `/launch` classique, c'est un bonus optionnel — pas un prérequis, le pilote headless du §4 suffit pour la science.

7. Protocole expérimental pour la question du rebond

- **Contrôle apparié par graine.** Deux runs de même graine, mêmes paramètres initiaux, l'un avec un journal d'interventions vide (contrôle), l'autre avec une intervention à t_0 (traitement) : les deux trajectoires sont **rigoureusement identiques jusqu'à t_0** (même séquence RNG consommée dans le même ordre jusque-là) et ne divergent qu'à partir de l'intervention. C'est ce qui rend la différenciation appariée légitime et peu coûteuse — s'appuyer dessus comme estimateur plutôt qu'en inventer un autre. **Utiliser le snapshot à t_0 du §4** pour brancher contrôle et traitement(s) sans re-simuler depuis $t = 0$ à chaque fois.
- **Plan minimal** : à t_0 choisi après relaxation (temps caractéristique du système, pas une valeur arbitraire). **Ne pas relancer de mesure de temps de relaxation à partir de zéro** : M4.3 a déjà estimé ce temps à sa baseline (`m4_3_credit_soc/scripts/renewal_relaxation_all_runs.py`, résultats cités dans son rapport final) — partir de cette estimation existante pour fixer t_0

(avec une marge de sécurité raisonnable), et ne remesurer que si les paramètres retenus pour §7 s'écartent trop de la baseline M4.3 pour que l'estimation reste fiable. Comparer ensuite (a) contrôle, (b) portée **fraction** (ou **new**, §1) à φ fixé sur A (levier primaire, §1, régime (b) du §3.1), (c) portée **all** de même amplitude sur A comme référence « hausse globale », (d) au moins une variante faisant varier γ pour placer la paire dans le régime (c) du §3.1 — ceci exerce délibérément le chemin coûteux de l'institution, pas seulement le chemin fermé. Plusieurs graines par cellule (le nombre exact à ta discrétion, mais jamais une seule). Optionnellement, des jeux différents de $\lambda/\delta/\sigma$ peuvent servir de régimes de population/marché secondaires (§2).

- **Fenêtre d'observation** : mesurer `prod_tot` sur une fenêtre post- t_0 dont la longueur est justifiée par le temps de relaxation du système, pas fixée arbitrairement.
- **Verdict à rendre**, avec la discipline fait/inférence/hypothèse/incertitude du reste du dépôt : effet rebond observé / non observé / non tranchable avec le budget de calcul disponible (§1 — l'objectif de cette itération est l'observation, pas l'attribution causale).
- Réutiliser l'outillage statistique existant du dépôt (`m4_3_credit_soc/scripts/lib_metrics.py` et voisins) si pertinent plutôt que le récrire ; le dire si rien ne s'applique.

8. Tests exigés

- **Institution de principal** (§3) : régime (a) reproduit $(K_\ell - K_b)/2$ à une tolérance documentée (pas nécessairement bit-exacte, §3.6) ; régime (b) vérifié exactement contre la formule fermée (rapport éq.14) ; régime (c) vérifié par comparaison Newton-exact vs le chemin choisi (chaud/tiède/froid), erreur sous la tolérance documentée.
- **Rejeu/déterminisme** (§4) : le journal source de ce test doit provenir d'une **session en direct réelle**, où les interventions (au moins une par portée) ont été soumises de façon asynchrone pendant que la boucle tournait — pas d'un plan headless écrit et exécuté par le même code que celui qui doit être vérifié. Ce journal, rejoué en mode headless, doit reproduire une trajectoire strictement identique à l'originale, en particulier même t effectif que celui enregistré.
- **Divergence à la reprise** (§4) : pour la fonctionnalité de reprise depuis un run M4.3 stocké, comparer systématiquement la série rejouée à la série originale et vérifier que l'écart mesuré est rapporté, pas seulement calculé en silence.
- **Invariant de pause** (§2) : N pas avec une ou plusieurs pauses au milieu produisent la même trajectoire que N pas sans pause.
- **Sémantique de portée**, au moins un test unitaire par portée : **new** ne change rien aux entités déjà nées ; **all** change bien toutes les entités vivantes au pas suivant ; **fraction** ne touche que le nombre attendu d'entités **et** ne change pas le défaut utilisé par les naissances postérieures à t_0 (vérifier explicitement qu'une entité née après t_0 reçoit l'ancien défaut, pas la valeur tirée).
- **Solvabilité**, si la piste du §3.4 est implémentée : le taux candidat $r(p, \Delta, \textit{principal})$ reste positif et ne provoque pas un pic de défauts mécanique au pas suivant, sur un échantillon de paires représentatif.
- Assertions Python simples, cohérent avec la convention du dépôt (pas de `pytest`, `CLAUDE.md`).

9. Livrables

- Code complet et fonctionnel dans `m4_3live_credit_soc/` (§10 pour l'arborescence attendue)
 - un système qui tourne réellement, pas un squelette : démarrer le serveur, piloter une session en direct dans un vrai navigateur, vérifier visuellement que les graphiques et le panneau de contrôle fonctionnent, avant de déclarer la tâche terminée.
- **Rapport de conception**, L^AT_EX compilé en PDF (`report/conception_m4_3live.pdf`) :

- toutes les décisions d'architecture de ce prompt justifiées avec citations `fichier.py:ligne` — en particulier les trois régimes du §3.1, l'architecture hot/warm/cold retenue (§3.2) et ses seuils recalibrés, le choix de borne institutionnelle (§3.2, dernier point), le statut du chantier taux/p (§3.4, abouti ou non), le caractère indépendant du fork (§3.5), et les résultats de la vérification de divergence à la reprise (§4) — y compris les décisions explicitement laissées ouvertes (§11) et la façon dont tu les as tranchées.
- **Rapport de résultats**, L^AT_EX compilé en PDF (`report/rapport_final.pdf`) : protocole du §7, résultats, figures, verdict d'observation sur l'effet rebond, limites — même discipline scientifique que `m4_3_credit_soc/report/rapport_final.tex`.
- Compilation PDF vérifiée (le `.tex` seul ne suffit pas).
- Un `README.md` et un `JOURNAL.md` courts dans `m4_3live_credit_soc/`, cohérents avec la convention du dépôt.

10. Arborescence attendue

```
m4_3live_credit_soc/
+-- prompts/PROMPT_M4_3LIVE.md      (ce document, source markdown)
+-- prompts/PROMPT_M4_3LIVE.tex/.pdf (cette version LaTeX/PDF)
+-- prompts/rapport_architecture_offline_online.pdf (etude du S3, lecture obligatoire)
+-- m4_3live/                        (paquet moteur INDEPENDANT, S3.5)
|   +-- model.py                     (institution du S3, per-entite A/gamma)
|   +-- live.py                      (session, boucle, journal, S4)
+-- driver/                          (pilote headless, S4 -- nom a ta discretion)
+-- web/ ou extension de simulation_lab/live/ (S6, a trancher S11)
+-- tests/
|   +-- test_institution.py           (S8, trois regimes)
|   +-- test_replay.py
|   +-- test_scopes.py
|   +-- test_resume_divergence.py     (S4, S8)
+-- scripts/                          (campagne du S7)
+-- results/                          (runs de la campagne, snapshots)
+-- report/
|   +-- conception_m4_3live.tex/.pdf
|   +-- rapport_final.tex/.pdf
|   +-- figures/
+-- README.md
+-- JOURNAL.md
```

11. Décisions laissées ouvertes, à trancher et documenter toi-même

- Nom exact du paquet/dossier si `m4_3live_credit_soc/` `m4_3live` s'avère gênant en pratique — réglé par défaut, changement possible si justifié.
- Extension de `simulation_lab/web/app.py` en place (isolée, §6) vs serveur HTTP dédié entièrement séparé pour `/live` — les deux sont acceptables, documenter le choix et pourquoi.
- Mécanisme exact de sondage HTTP (intervalle, format JSON) — reprendre la convention `app.js` existante par défaut, l'adapter si le débit d'un direct l'exige, en le disant.
- Étendre l'hétérogénéité par entité à d'autres paramètres que A/γ (ex. `delta` par entité) — hors mandat par défaut, seulement si le protocole du §7 en révèle un besoin net.
- **Co-variation de K_0 avec une intervention A/γ** (§2) — automatique, optionnelle sur demande, ou délibérément découplée ; à trancher avant la campagne du §7 et à documenter, quelle que soit la réponse.

- **Borne supérieure du transfert optimal** (§3.2, égalisation des capitaux vs optimum général $h(C)$) — choix institutionnel, pas numérique.
- **Seuils hot/warm/cold** (§3.2) — recalibrer sur la machine cible, ne pas reprendre tels quels les chiffres du banc synthétique du rapport.
- **Statut du chantier taux/ p** (§3.4) — abouti (avec l’hypothèse de conversion $\Delta \rightarrow r$ documentée et validée) ou explicitement ajourné au profit de `_pair_rate`, mais pas laissé dans un état intermédiaire non documenté.

12. Consignes de méthode de travail, pour toi (Opus 5)

- **Livrer un système complet, pas des ébauches.** Chaque pièce du §10 doit fonctionner de bout en bout avant que la tâche soit déclarée terminée — pas de squelette avec des TODO, pas de fonctionnalité à moitié câblée.
- **Calibrer la longueur des livrables écrits sur leur substance.** Les deux rapports L^AT_EX doivent couvrir ce que ce prompt demande, sans sections de remplissage, sans résumé redondant en fin de document, sans paragraphe qui répète ce qui vient d’être dit autrement.
- **Pas de vérification en boucle au-delà de ce qui est demandé.** Les tests du §8 et le test manuel dans un vrai navigateur (§9) sont les vérifications exigées — inutile d’ajouter des passes de relecture supplémentaires au-delà.
- **Périmètre : livrer ce qui est demandé, à l’échelle demandée.** Prendre seul les décisions d’implémentation de routine (y compris celles listées en §11); ne pas élargir le mandat scientifique au-delà du §7; si une lecture différente de ce prompt changerait matériellement le travail à produire, le signaler en une phrase et continuer avec la lecture la plus proche du texte plutôt que de redéfinir la tâche.

Sous-agents : subdiviser intelligemment, pas systématiquement

Ce dépôt et cette tâche se prêtent à de la délégation — utiliser les sous-agents avec discernement plutôt que tout faire en séquence toi-même ou, à l’inverse, tout déléguer par réflexe :

- **Réserver ton propre raisonnement (Opus) aux décisions qui déterminent la validité scientifique et architecturale du travail :** l’institution de principal et ses trois régimes (§3), la sémantique exacte des trois portées (§2), le protocole expérimental et l’écriture du verdict (§7), l’intégration avec `simulation_lab` sans régression (§6), et la relecture finale de cohérence entre code et rapports. Ce sont les endroits où une erreur de jugement invalide tout le reste.
- **Déléguer les missions simples, bien délimitées et vérifiables isolément à des sous-agents sur un modèle plus léger (Sonnet ou Haiku) :** écrire le CSS/JS répétitif de l’IHM une fois la structure de l’API fixée, écrire un test unitaire donné une spécification précise (§8), lancer et collecter une graine de la campagne du §7 une fois le protocole gelé, rédiger une section descriptive du rapport de conception à partir de décisions déjà prises et citées, exécuter une vérification de non-régression sur `simulation_lab` et rapporter le résultat brut.
- Donner à chaque sous-agent délégué un mandat autonome et complet (pas un fragment de phrase) : quel fichier, quel contrat d’entrée/sortie, quel test de validation — puisqu’un sous-agent frais ne voit pas ce prompt, lui fournir directement les extraits pertinents (numéros de ligne, formules, sémantique de portée) plutôt que d’y renvoyer par référence.
- Ne pas déléguer la vérification de ton propre travail à un autre sous-agent en plus des tests déjà exigés (§8/§9) — pas de double passe de relecture par sous-agent interposé.
- Ne pas ouvrir plus de sous-agents que la structure du travail n’en justifie — une tâche qui tient en quelques appels d’outils directs n’a pas besoin d’être déléguée.